Studio Nx

Backend(FastAPI/Python) 11 — ProjectMember 모델 컬럼

2026-10-12

Backend(FastAPI/Python) 11 — `ProjectMember` 모델 컬럼

항목	내용
파일	`studionx/backend/app/models/project.py` (`ProjectMember` 클래스)
작업	P1-03 백엔드 공통: ORM 모델 컬럼 (`docs/DB.md` §3.1, `studionx/db/01_phase1_account.sql`)
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	10 `Project` 모델 · 12 `Track` 모델

1. 하는 일

MySQL의 `project_members` 테이블(어느 회원이 어느 프로젝트에 어떤 권한으로 참여하는지)을 파이썬 클래스 `ProjectMember`로 옮겨 적습니다. 기본 키와 FK 2개는 뼈대에 이미 있었고, 이번에 나머지 3칸과 권한 목록 `MemberRole`을 채웠습니다.

project_id	user_id	role	뜻
12	7	`OWNER`	7번 회원이 12번 프로젝트를 만들었습니다
12	9	`EDITOR`	9번 회원은 12번 프로젝트를 고칠 수 있습니다 (Phase 6 협업)

`role`	뜻
`OWNER`	만든 사람입니다. 삭제와 초대를 할 수 있습니다
`EDITOR` (DB 기본값)	편집할 수 있습니다
`VIEWER`	보기만 할 수 있습니다

2. 언제 쓰나

쓰는 곳	하는 일
P1-13 프로젝트 만들기 (myProject.html의 **New project**)	프로젝트와 함께 `OWNER` 행 하나를 넣습니다 (FR-122)
P1-13, P1-16 목록	"내가 멤버인 프로젝트"를 찾을 때 이 테이블을 거칩니다
Phase 6 협업	초대받은 회원을 `EDITOR`나 `VIEWER`로 추가하고, 톡방에서 읽은 위치 (`last_read_message_id`)를 기록합니다

Phase 1에서는 **프로젝트마다 OWNER 한 행만** 씁니다.

3. 코드

import (파일 맨 위)

```
from datetime import datetime
from decimal import Decimal
from enum import StrEnum
from typing import Any
```

```
from sqlalchemy import (
    JSON,
    BigInteger,
    Enum,
    FetchedValue,
    ForeignKey,
    Integer,
    Numeric,
    String,
    text,
)
from sqlalchemy.dialects.mysql import DATETIME, TINYINT
from sqlalchemy.orm import Mapped, mapped_column

from app.db.session import Base
```

`project.py` 맨 위의 import입니다 (`Project`와 함께 씁니다). 이번에 **`StrEnum`과 `Enum`**을 추가했습니다. 줄이 길어져서 `ruff`가 괄호로 묶고 한 줄에 하나씩 적었습니다. `ProjectMember`는 이 중 `StrEnum`, `BigInteger`, `Enum`, `ForeignKey`, `text`, `DATETIME`, `Mapped`, `mapped_column`, `Base`를 씁니다.

함수 (클래스와 칸 선언)

```
class MemberRole(StrEnum):
    OWNER = "OWNER"
    EDITOR = "EDITOR"
    VIEWER = "VIEWER"


class ProjectMember(Base):
    __tablename__ = "project_members"

    # 복합 기본 키 (project_id, user_id)
    project_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("projects.project_id", ondelete="CASCADE"), primary_key=True
    )
    user_id: Mapped[int] = mapped_column(BigInteger, ForeignKey("users.user_id"), primary_key=True)

    role: Mapped[MemberRole] = mapped_column(Enum(MemberRole), server_default=MemberRole.EDITOR)
    last_read_message_id: Mapped[int | None] = mapped_column(BigInteger)  # Phase 6 톡방
    joined_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)")
    )
```

SQL과 비교

SQL (`01_phase1_account.sql`)	파이썬
`PRIMARY KEY (project_id, user_id)`	두 칸 모두 `primary_key=True`
`FOREIGN KEY (project_id) REFERENCES projects (project_id) ON DELETE CASCADE`	`ForeignKey("projects.project_id", ondelete="CASCADE")`
`role ENUM('OWNER','EDITOR','VIEWER') NOT NULL DEFAULT 'EDITOR'`	`Enum(MemberRole)`, `server_default=MemberRole.EDITOR`
`last_read_message_id BIGINT UNSIGNED NULL`	`Mapped[int | None]` (FK 없음: DB에도 없습니다)

SQL (01_phase1_account.sql)	파이썬
`joined_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3)`	`server_default=text(…)`

문법 풀이

import 문법	뜻
`from 모듈 import (A, B, …)`	가져올 이름이 많을 때 괄호로 묶어 여러 줄에 적습니다. 한 줄로 적은 것과 같습니다

식	뜻
복합 기본 키	두 칸을 **합쳐서** 한 행을 가리킵니다. `(12, 7)`은 한 번만 있을 수 있고, `(12, 9)`와 `(13, 7)`은 따로 있을 수 있습니다
`db.get(ProjectMember, (12, 7))`	복합 기본 키로 찾을 때는 값을 튜플로 넘깁니다
`ondelete="CASCADE"`	가리키는 프로젝트 행이 지워지면 **DB가** 이 행도 함께 지웁니다

4. 흐름 (프로젝트를 만들 때, P1-13)

1. `Project(owner_id=7, title=…)`를 넣고 `flush`해서 `project_id=12`를 받습니다.
2. `ProjectMember(project_id=12, user_id=7, role=MemberRole.OWNER)`를 넣습니다.
3. DB가 `joined_at`을 채웁니다.
4. 한 번에 `commit`합니다. 중간에 실패하면 둘 다 들어가지 않습니다 (FR-122).

5. 왜 이렇게

선택	이유
기본 키를 `(project_id, user_id)`로	같은 회원이 같은 프로젝트에 두 번 들어갈 수 없습니다. 그래서 따로 번호 칸이 필요하지 않습니다
`project_id`에 `ON DELETE CASCADE`	프로젝트 행이 정말 지워지면 멤버 행만 남을 이유가 없습니다. 보통 삭제는 `deleted_at`을 기록하는 소프트 삭제라서 이런 경우는 드뭅니다 (관리자가 DB에서 직접 지울 때 등)
`user_id`에는 `CASCADE`가 없음	회원은 지우지 않고 `status=DELETED`로 바꿉니다 (08). 회원 행을 실수로 지우려 하면 DB가 막습니다
`role` 기본값이 `EDITOR`	Phase 6에서 초대받는 사람은 보통 편집자이기 때문입니다. 만든 사람은 코드에서 반드시 `OWNER`를 적습니다
`last_read_message_id`에 FK가 없음	DB에도 없습니다. 가리킬 `chat_messages`는 Phase 6에서 만들고, 메시지가 지워져도 읽은 위치 숫자는 남아도 됩니다

6. 시험

`tests/test_project_member_model.py` **(새 파일, 5개)**: 실제 MySQL로 확인했습니다. 넣은 행은 시험이 끝나면 되돌립니다 (시험 뒤 확인: 0행).

시험	확인한 것
칸이 DB와 같은지	칸 5개, `NULL` 허용 여부, FK 2개(`project_id`는 `ON DELETE CASCADE`까지)가 같습니다

시험	확인한 것
기본 키	`["project_id", "user_id"]`
OWNER와 기본값	`role=OWNER`를 적으면 `OWNER`가 되고, 적지 않으면 DB가 `EDITOR`로 채웁니다. `joined_at`은 채워지고, `last_read_message_id`는 비어 있습니다
같은 회원을 두 번 넣음	DB가 거절합니다 (`IntegrityError`)
프로젝트 행을 지우면	멤버 수가 1에서 0이 됩니다 (DB의 `CASCADE`)

전체 시험 66개가 통과했고, `ruff` 검사도 통과했습니다.